

```
In [1]: import sys
import platform

print("Python version  :", sys.version.split()[0])
print("Interpreter path :", sys.executable)
print("Platform         :", platform.platform())
print()
print("Running in ai-lab:", "ai-lab" in sys.executable)
```

Python version : 3.11.16
 Interpreter path : C:\Users\DELL\.conda\envs\ai-lab\python.exe
 Platform : Windows-10-10.0.26200-SP0

Running in ai-lab: True

```
In [2]: print("Hello, Introduction to Artificial Intelligence")
```

Hello, Introduction to Artificial Intelligence

```
In [4]: import pandas as pd

print("pandas version:", pd.__version__)
```

pandas version: 3.0.5

```
In [5]: import pandas as pd

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"
df = pd.read_csv(url)

print("Shape (rows, columns):", df.shape)
df.head()
```

Shape (rows, columns): (891, 15)

```
Out[5]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	eml
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Sou
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Sou
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Sou
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Sou
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Sou

```
In [6]: df.head()
```

Out[6]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	eml
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Sou
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Sou
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Sou
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Sou

In [7]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
Column Non-Null Count Dtype
--- -
0 survived 891 non-null int64
1 pclass 891 non-null int64
2 sex 891 non-null str
3 age 714 non-null float64
4 sibsp 891 non-null int64
5 parch 891 non-null int64
6 fare 891 non-null float64
7 embarked 889 non-null str
8 class 891 non-null str
9 who 891 non-null str
10 adult_male 891 non-null bool
11 deck 203 non-null str
12 embark_town 889 non-null str
13 alive 891 non-null str
14 alone 891 non-null bool
dtypes: bool(2), float64(2), int64(4), str(7)
memory usage: 92.4 KB

In [8]: df.describe()

Out[8]:

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

In [9]: missing = df.isnull().sum()
missing = missing[missing > 0].sort_values(ascending=False)

print("Columns containing missing values:\n")

```
print(missing)
print()
print("As a percentage of all rows:\n")
print((missing / len(df) * 100).round(1))
```

Columns containing missing values:

```
deck          688
age           177
embarked       2
embark_town    2
dtype: int64
```

As a percentage of all rows:

```
deck          77.2
age           19.9
embarked       0.2
embark_town    0.2
dtype: float64
```

```
In [11]: # --- Exercise 2: your answer -----
# Use df.info() above, then fill in this dictionary from what you read.

missing_columns = {
    "deck": 688,
    "age": 177,
    "embarked": 2,
}

print("Columns with missing values (your answer):", missing_columns)
print()
print("Checked against pandas:")
print(df.isnull().sum()[df.isnull().sum() > 0])
```

Columns with missing values (your answer): {'deck': 688, 'age': 177, 'embarked': 2}

Checked against pandas:

```
age          177
embarked      2
deck          688
embark_town    2
dtype: int64
```

```
In [12]: numeric = df.select_dtypes(include="number")
cv = (numeric.std() / numeric.mean()).abs().sort_values(ascending=False)

print("Coefficient of variation (std / mean), largest first:\n")
print(cv.round(3))
print()
print("Largest:", cv.index[0])
```

Coefficient of variation (std / mean), largest first:

```
parch        2.112
sibsp        2.108
fare         1.543
survived     1.268
age          0.489
pclass       0.362
dtype: float64
```

Largest: parch

`parch` has the highest coefficient of variation (2.11), meaning its spread is more than twice its own mean — most passengers have 0 but a few have several, creating a long tail. A model without feature scaling would treat `parch`'s variance as artificially more important than an evenly-distributed column like `age`, purely because of scale.

In []: